Lab 1: Working with Schemas, Managed Tables, Volumes & DBUtils in Unity Catalog

Lab Name:

Working with Schemas, Managed Tables, Volumes & DBUtils in Unity Catalog

Lab Summary

In this lab, we:

- Create a custom catalog (`databricks-abic`)
- Create a Bronze schema
- Create a Managed Table using UI
- Create a Managed Volume
- Upload raw files into volume folders
- Query data directly from volumes
- Explore Databricks Notebooks
- Use DBUtils for file and parameter management

This lab establishes the Bronze layer (Raw Zone) using Unity Catalog.

Prerequisites

- Unity Catalog enabled workspace
- Permission to create catalog
- Serverless compute available
- Sales CSV files:

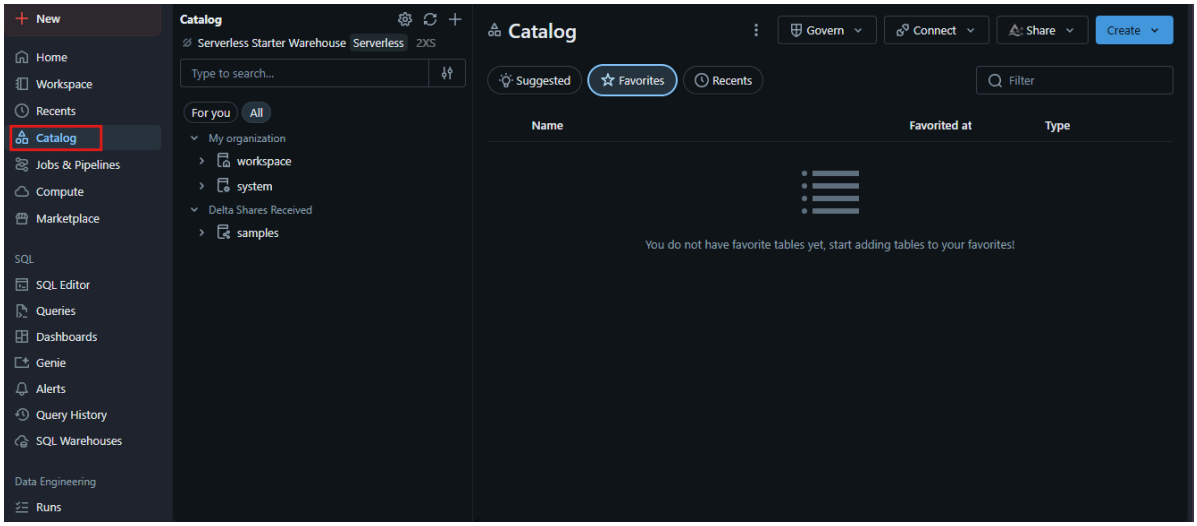
- fact_sales.csv
- dim_customers.csv
- dim_products.csv
- dim_stores.csv

Steps to Complete the Lab

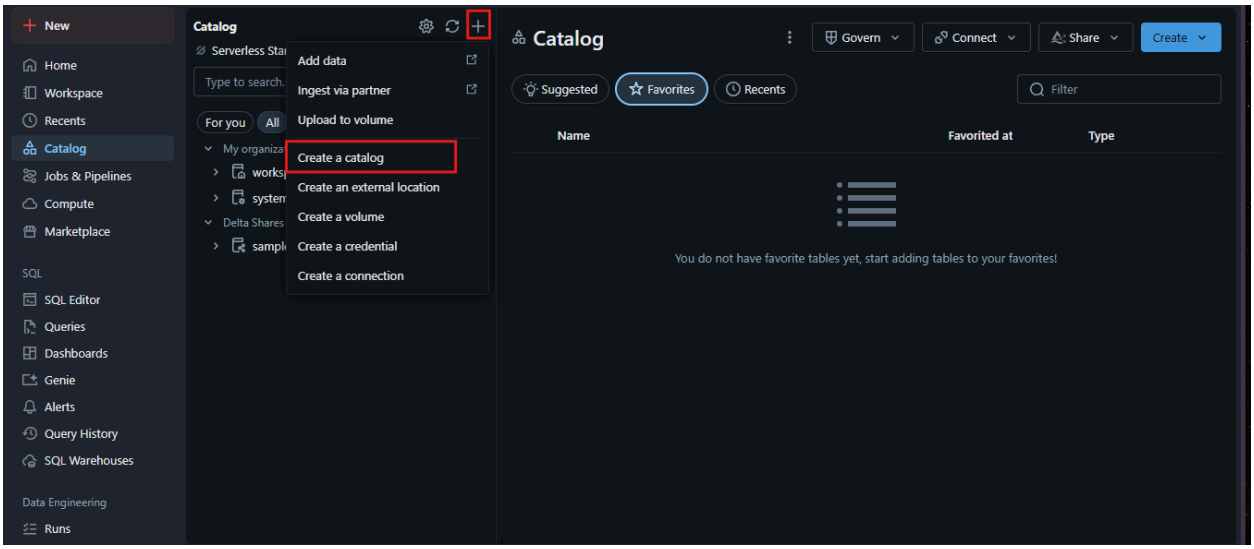
Task 1: Create Custom Catalog (via UI)

Step 1: Navigate to Catalog

- Click **Catalog** from left panel

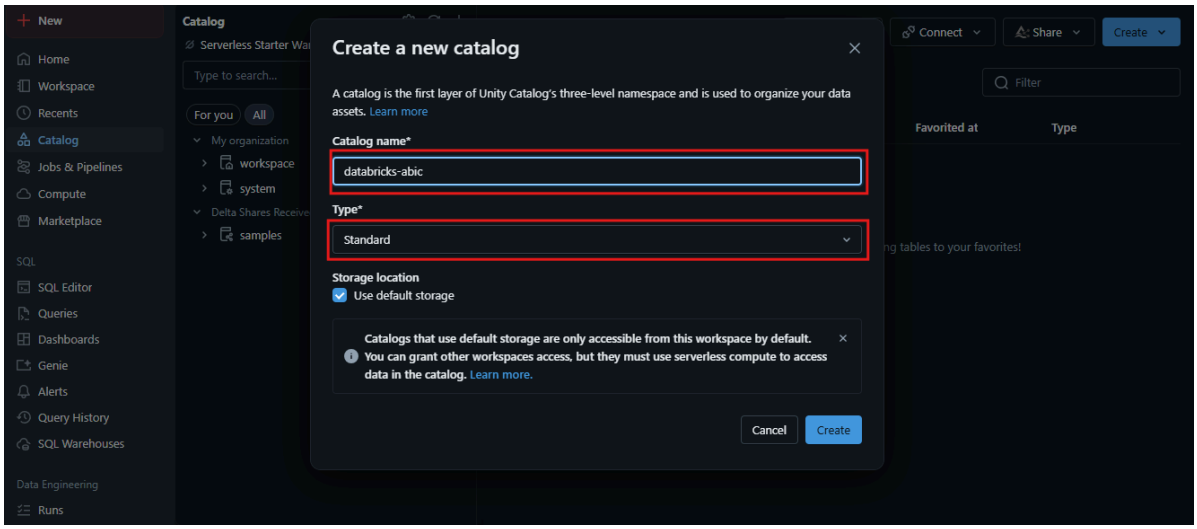


Step 2: Click “Create Catalog”



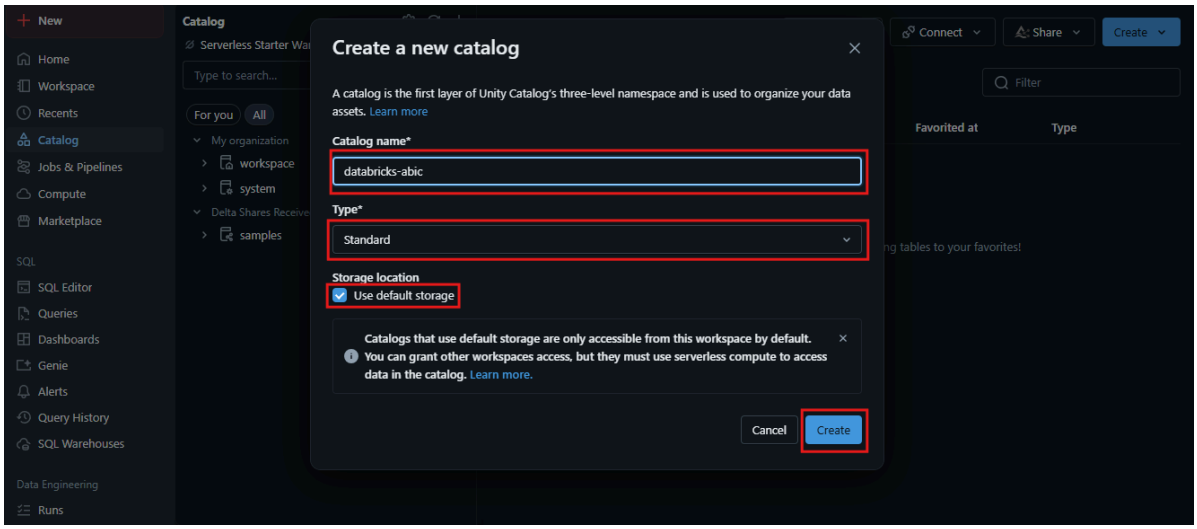
Step 3: Enter Catalog Details

- Catalog Name: `databricks-abic` —→ Type: Standard



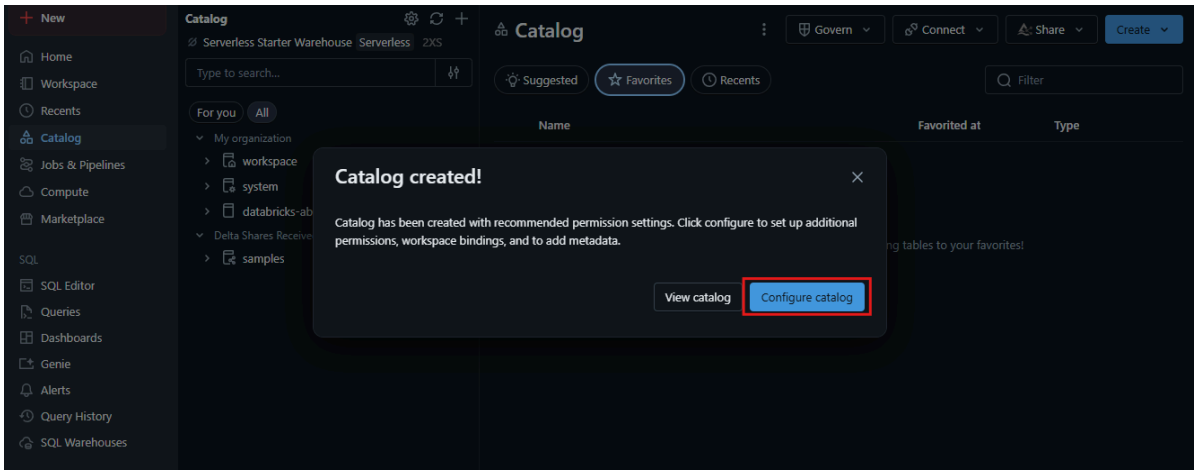
- Managed Storage Location: Leave blank —→ Click Create

  (Uses metastore default storage location)

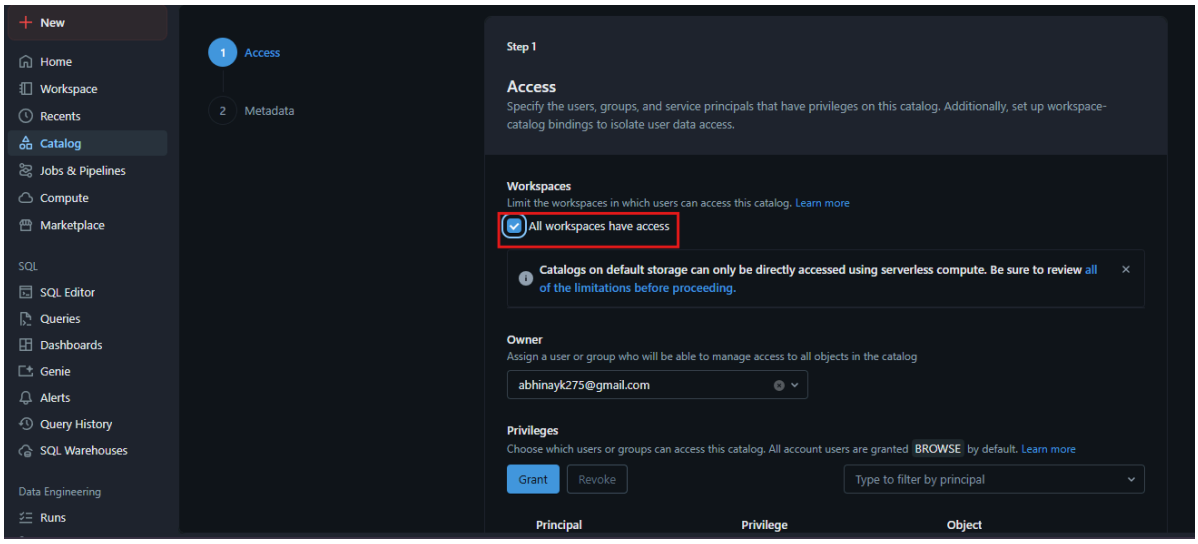


Step 4: Catalog Configuration

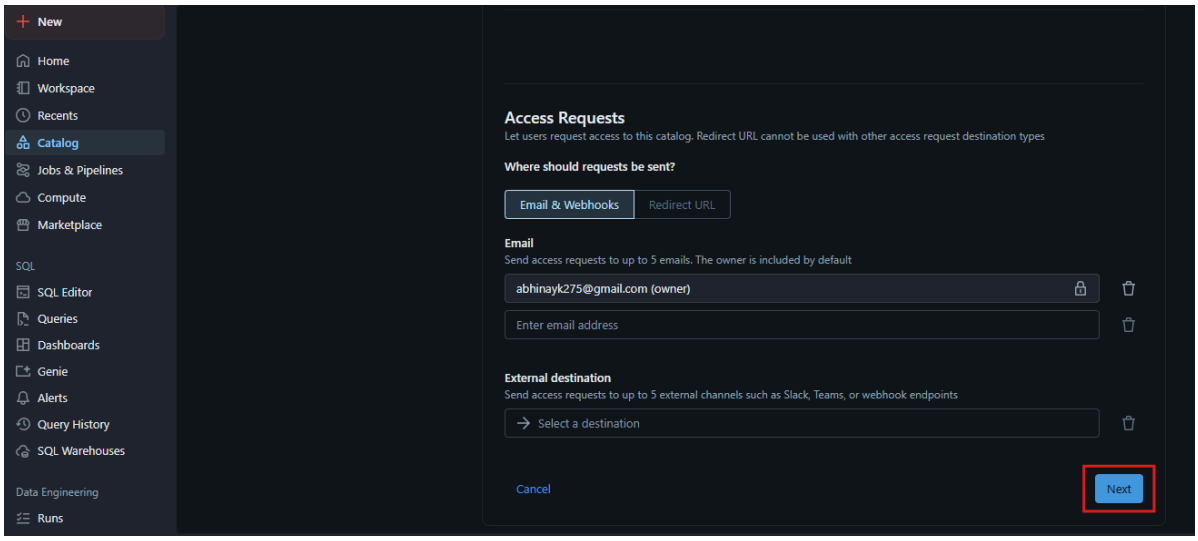
- Click Configure catalog



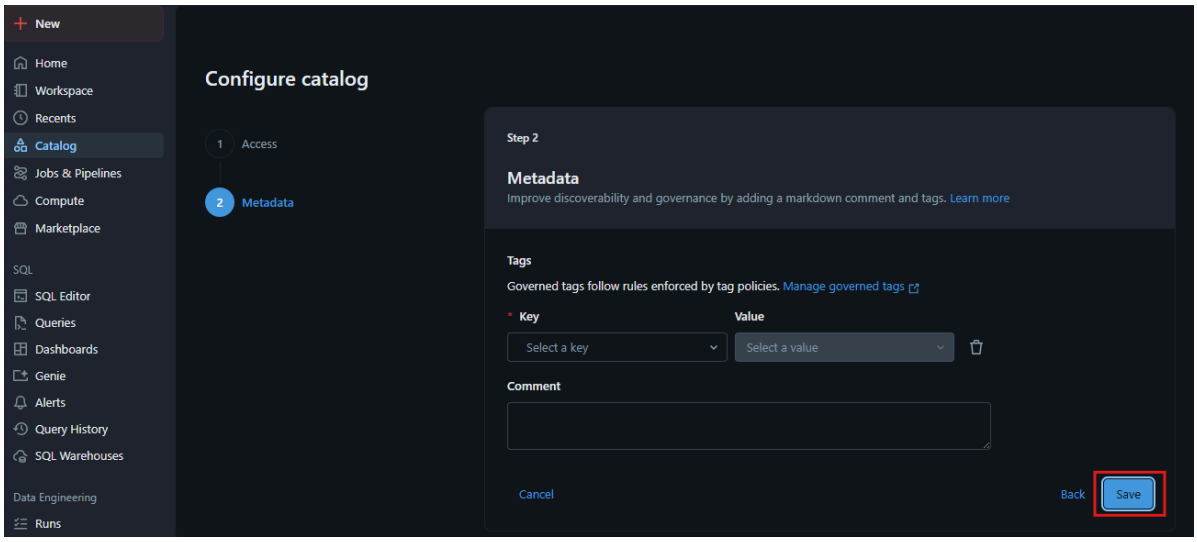
- Access: All workspaces



- Privileges: Leave default —→ Click Next

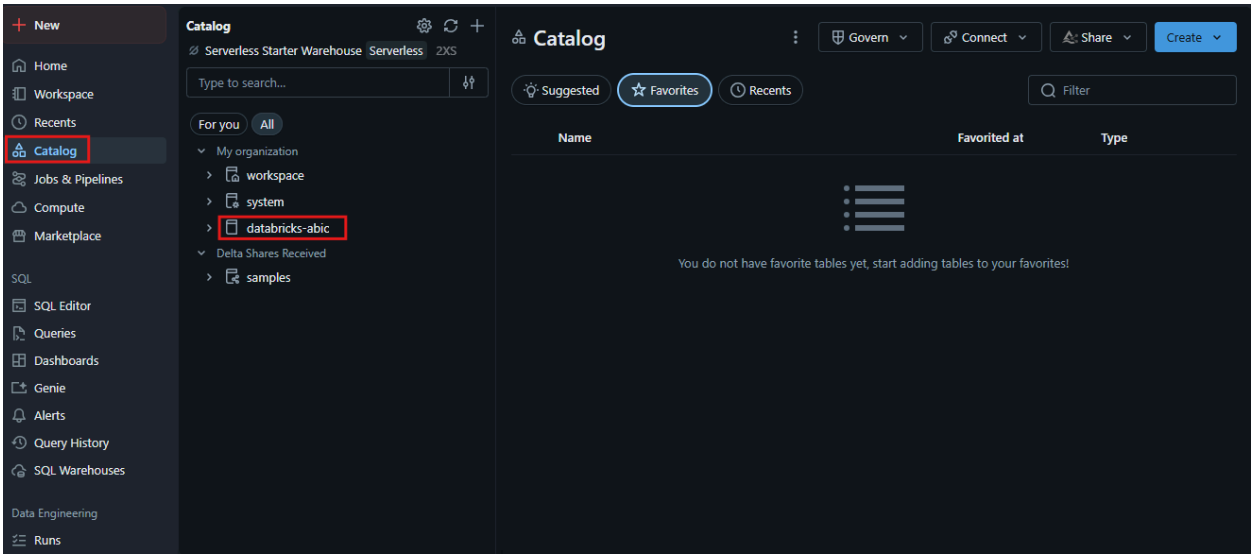


- Click Save

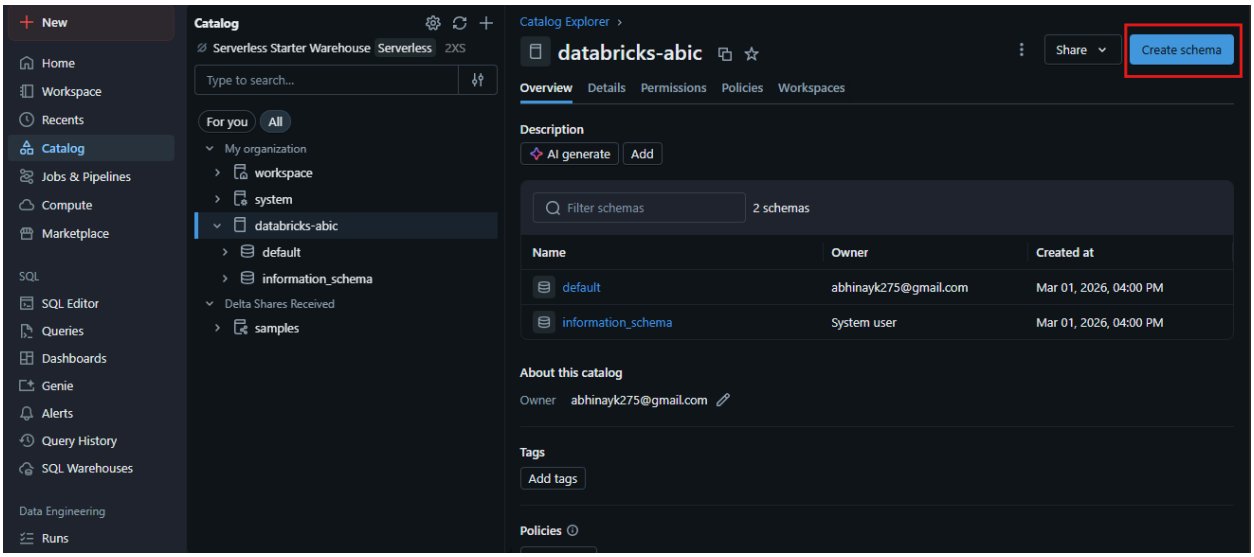


Task 2: Create Bronze Schema

Step 1: Open `databricks-abic` catalog

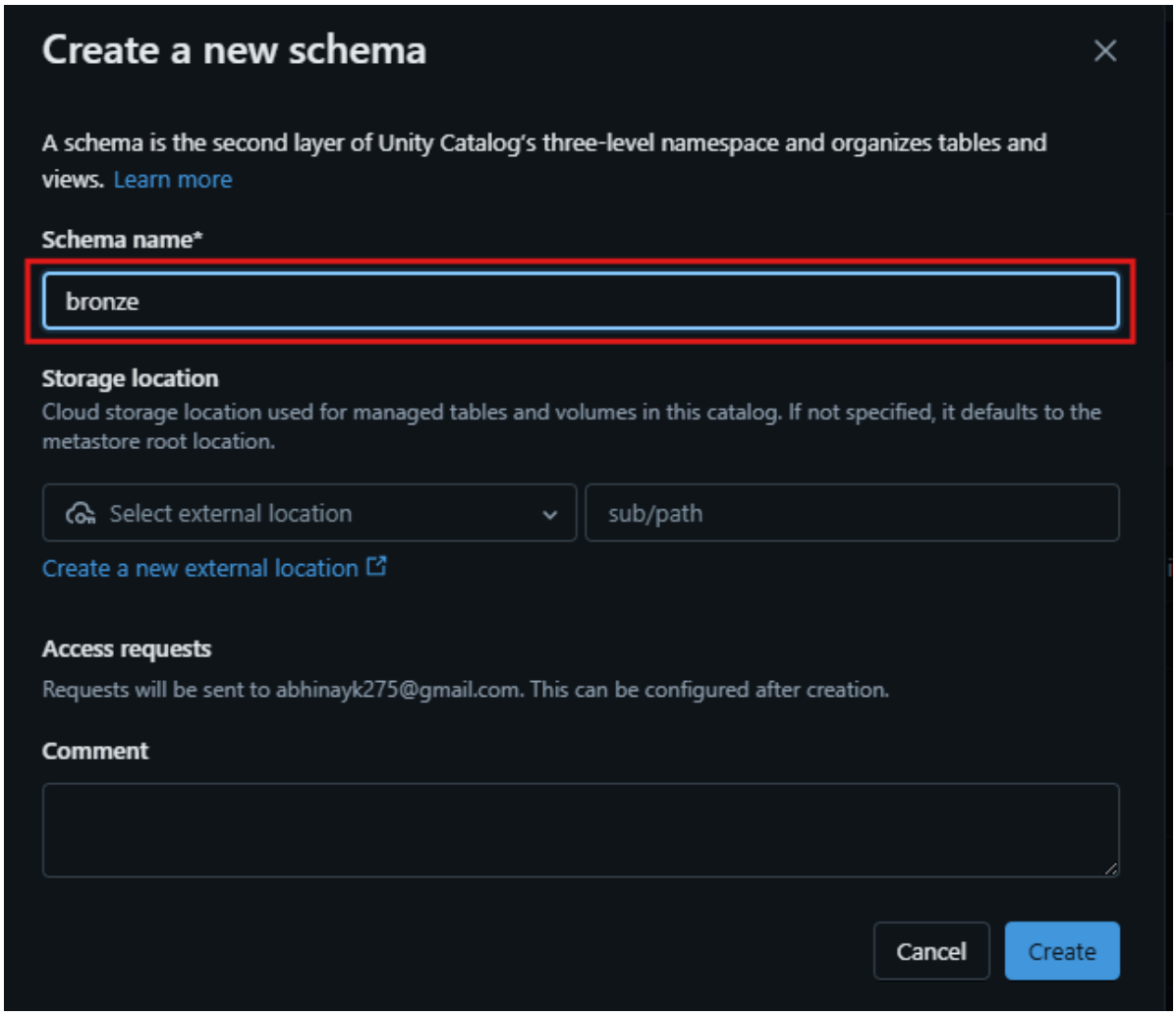


Step 2: Click → Create Schema



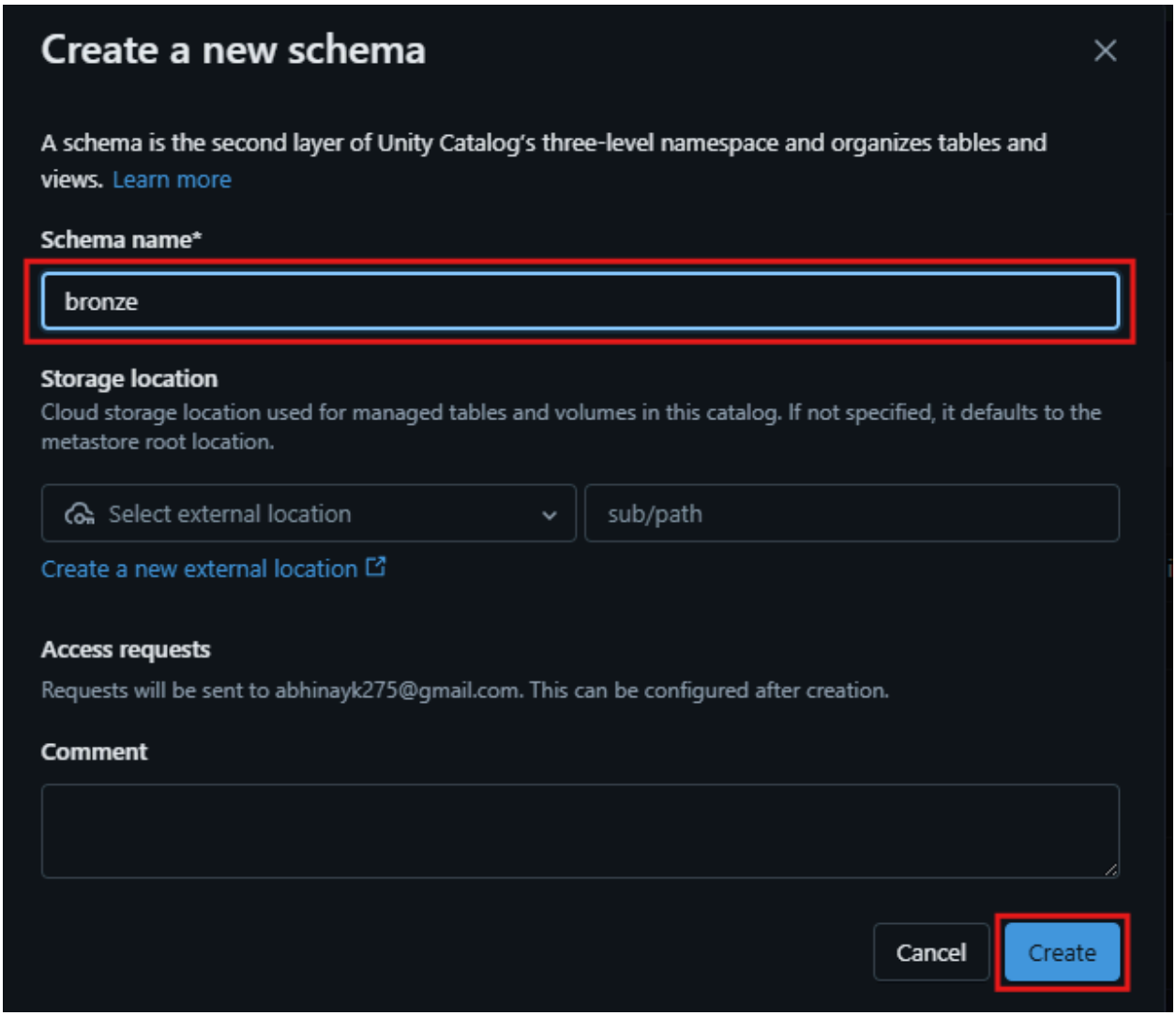
Step 3: Enter Details

- Schema Name: `bronze`



- Storage Location: Leave blank (inherits from catalog)

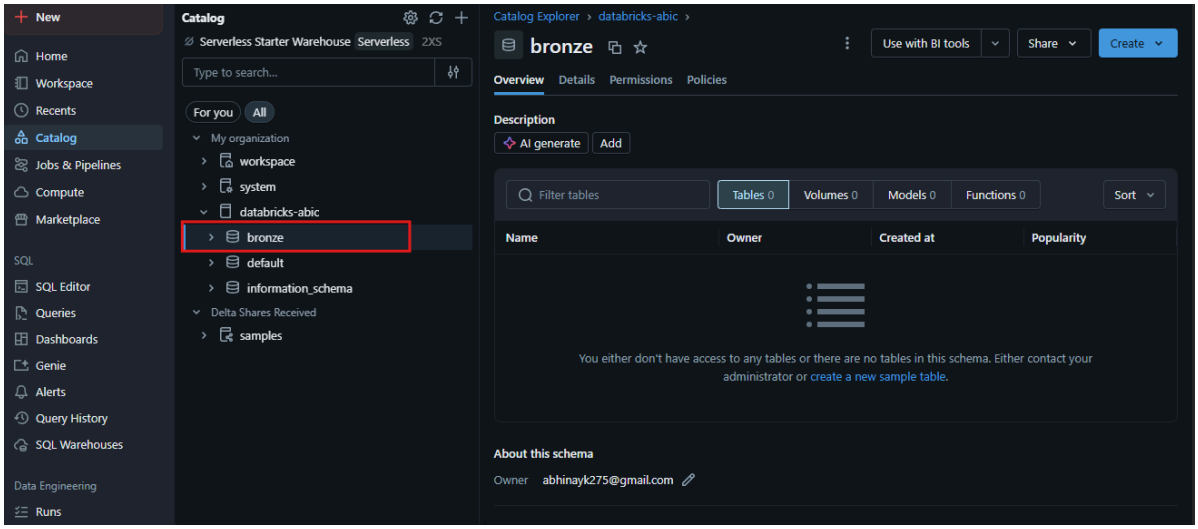
Step 4: Click Create



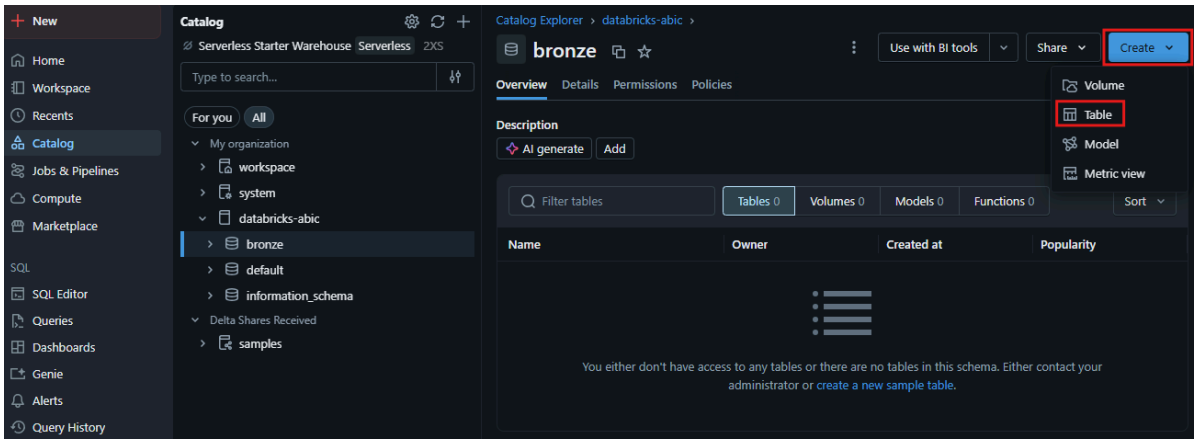
Task 3: Create a Managed Table using UI (Demo Purpose)

Step 1: Open Bronze Schema

- Click `databricks-abic.bronze`

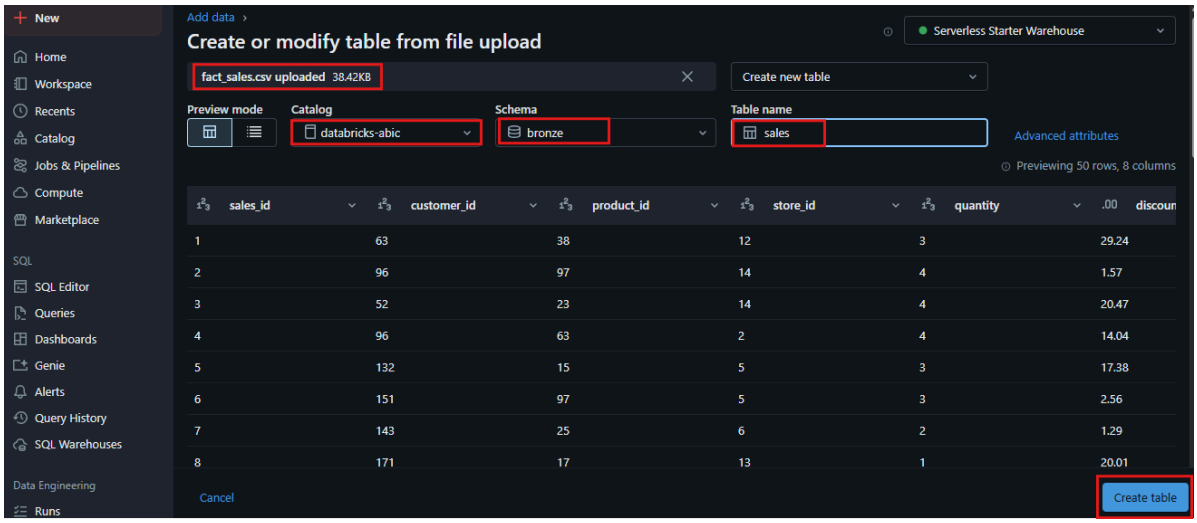


- Click **Create → Table**



Step 2: Upload CSV File

- Upload `fact_sales.csv`
- Table Name: `sales`
- Confirm catalog: `databricks-abic`
- Confirm schema: `bronze`
- Click **Create Table**



Important:

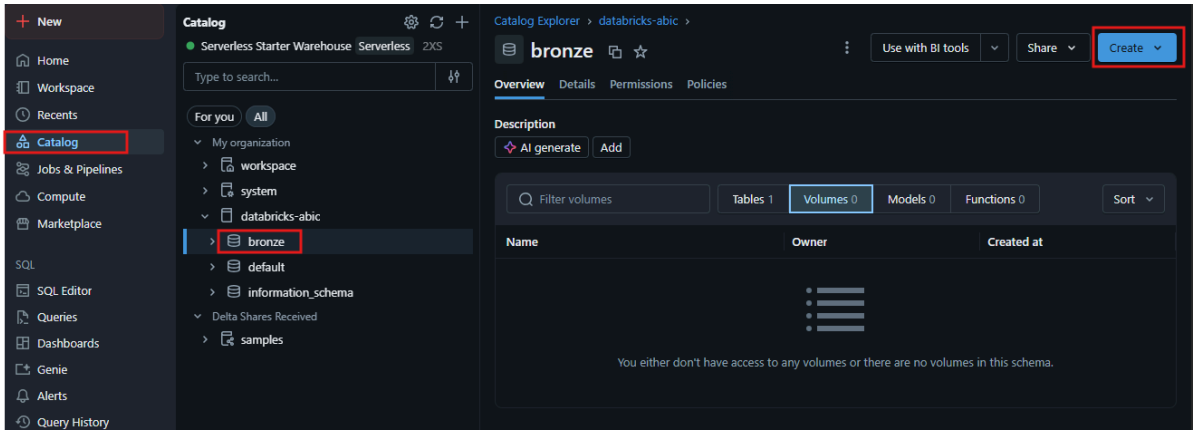
- This creates a Managed Table
- Data stored under metastore managed location
- File converted to Delta format

Task 4: Create Managed Volume

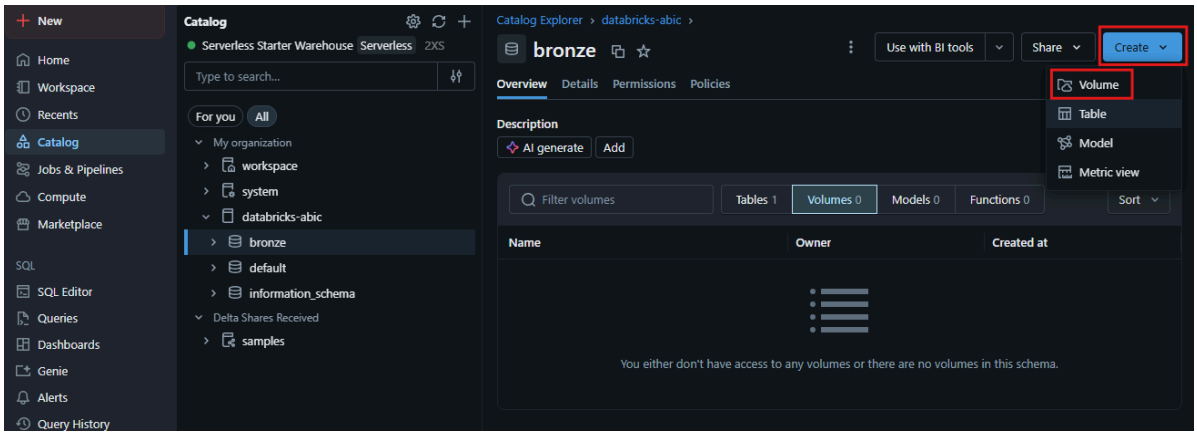
Method 1: Using UI

Step 1:

- Go to `databricks-abic.bronze`

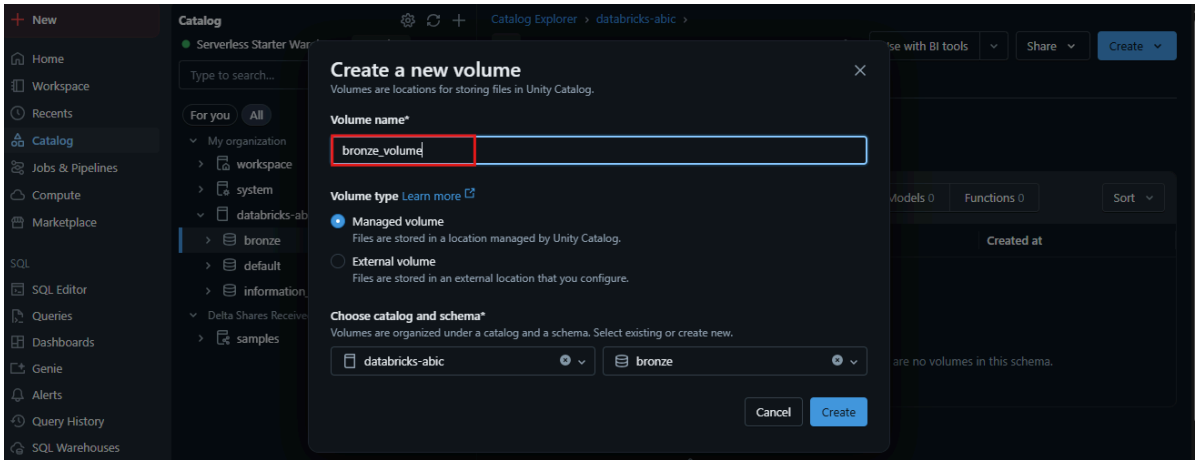


- Click **Create → Volume**

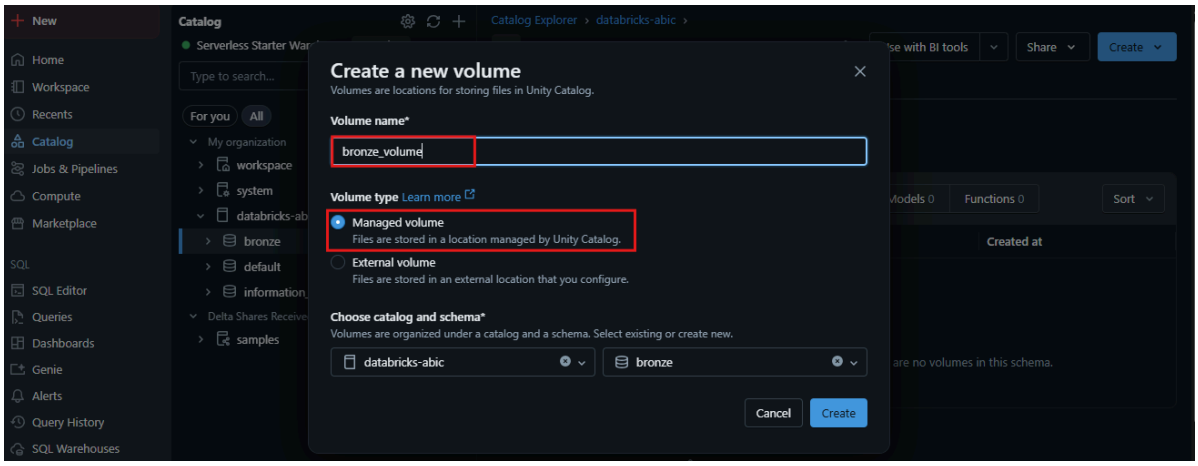


Step 2: Configuration

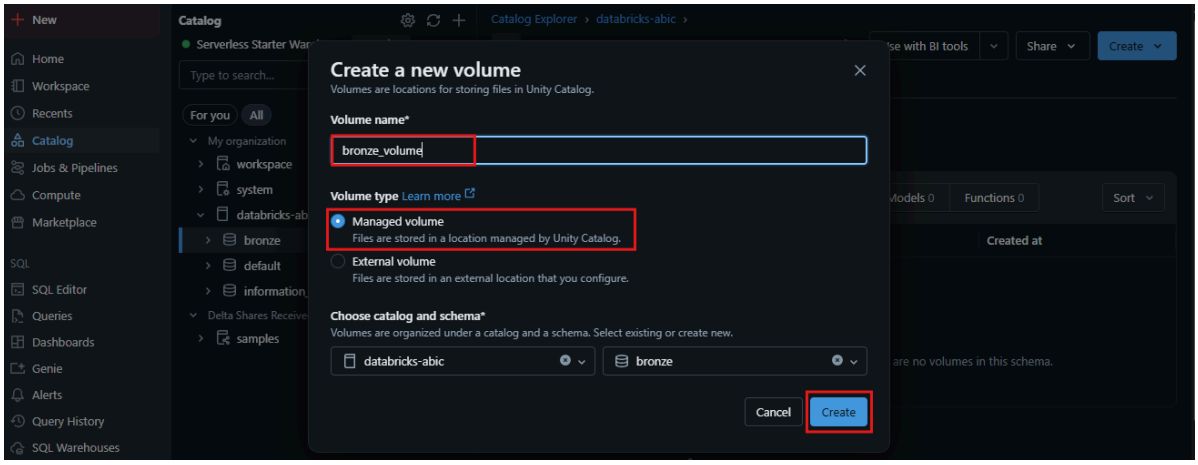
- Volume Name: `bronze_volume`



- Type: Managed

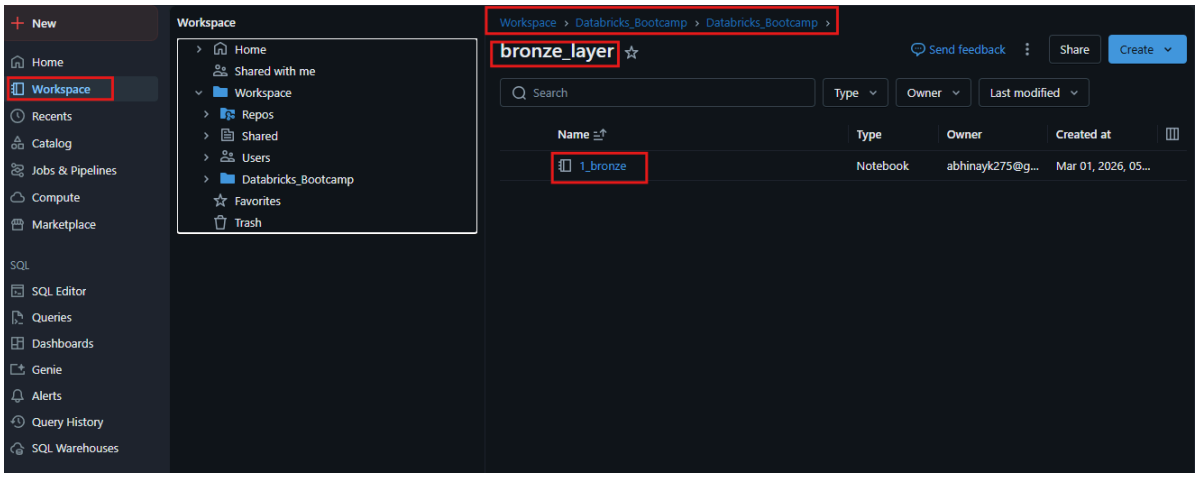


- Click Create



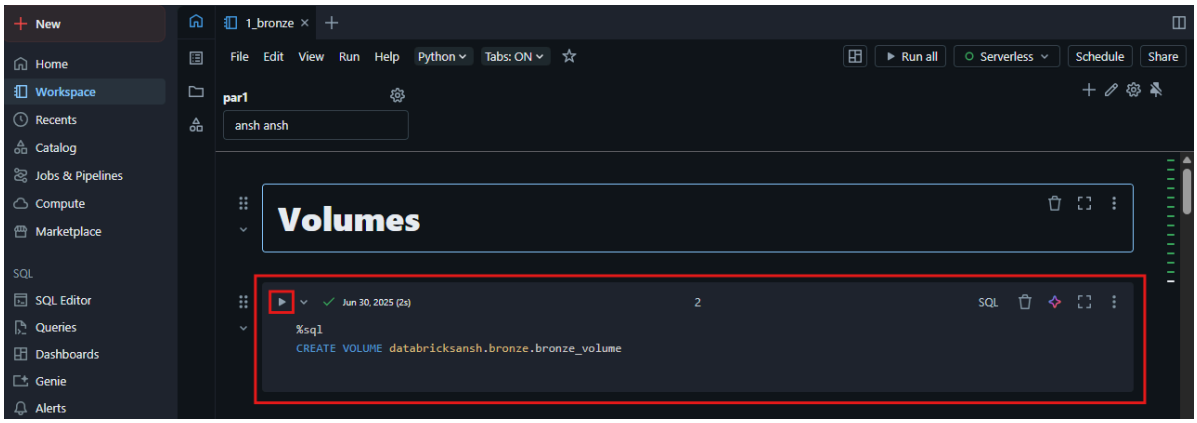
Method 2: Using SQL

- Go to workspace —→ Databricks_Bootcamp —→ bronze_layer —→ Notebook(1_bronze)

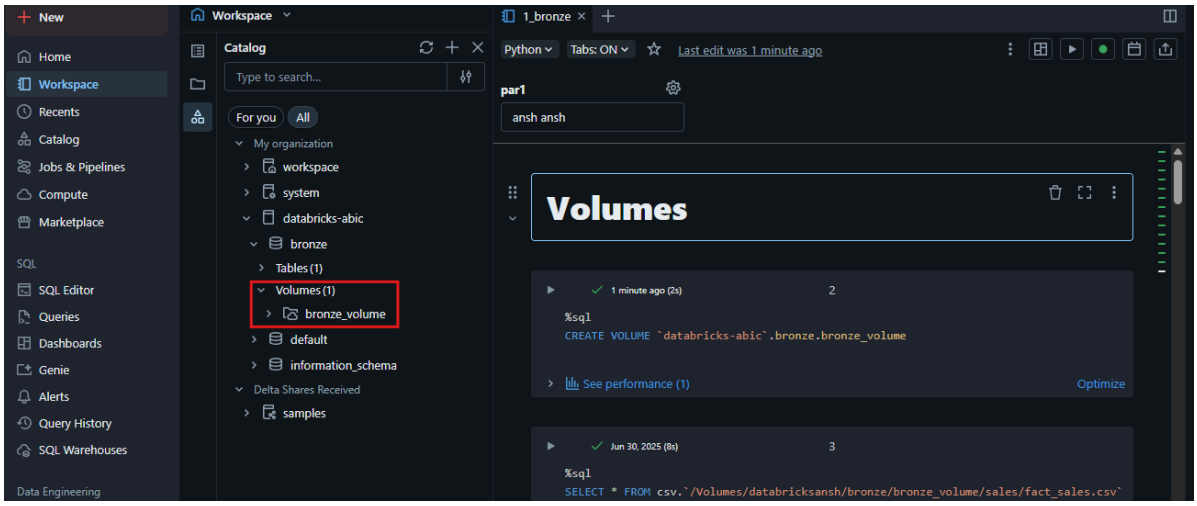


- Run the cell with code below:

```
CREATE VOLUME databricks-abic.bronze.bronze_volume;
```



- Refresh schema → Volume appears



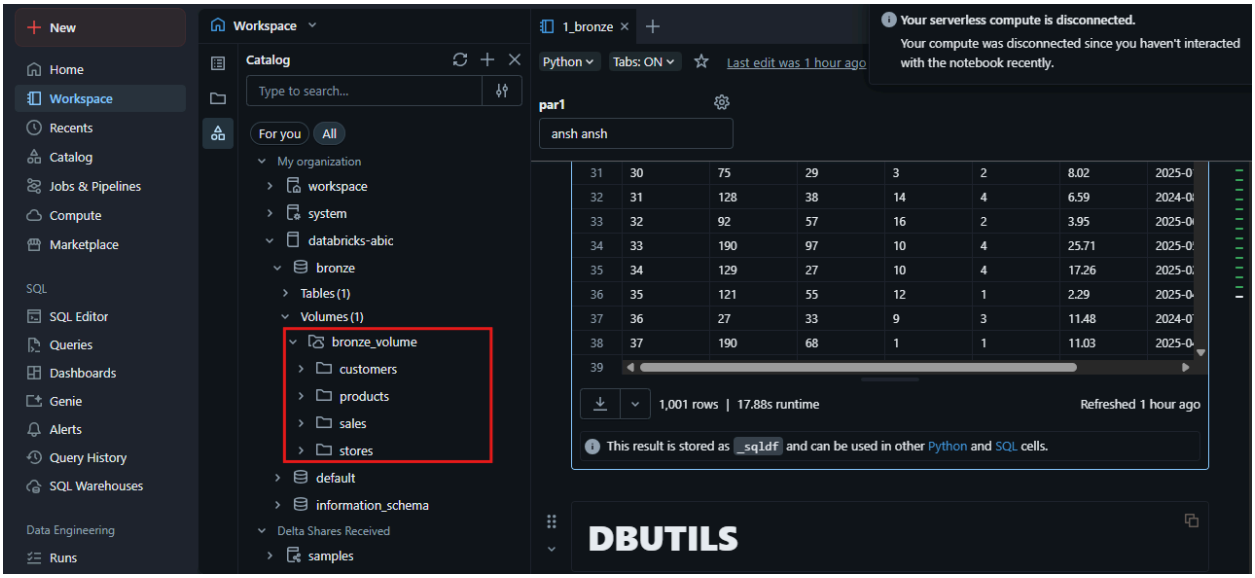
Task 5: Create Folder Structure inside Volume

Create directories:

- sales
- customers
- products
- stores

Path becomes:

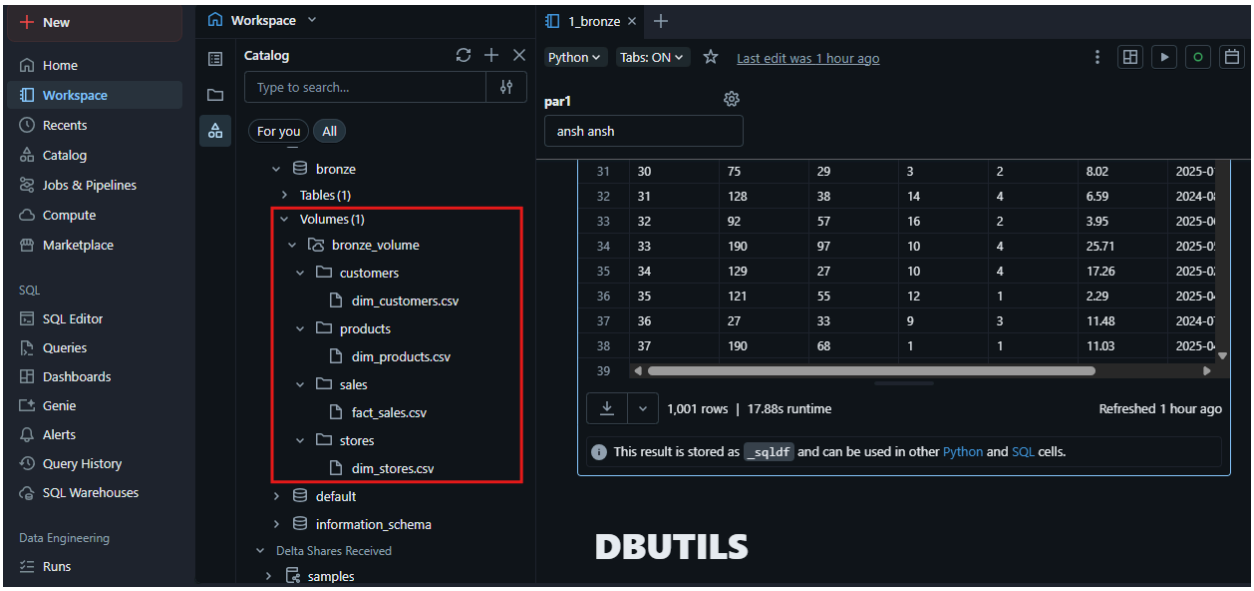
```
/Volumes/databricks-abic/bronze/bronze_volume/
```



Task 6: Upload Data into Volume

Upload:

- fact_sales.csv → sales/
- dim_customers.csv → customers/
- dim_products.csv → products/
- dim_stores.csv → stores/



Task 7: Query Data from Volume

Connect to Serverless compute.

Run:

```
SELECT *
FROM csv.`/Volumes/databrick-abic/bronze/bronze_volume/sales/
fact_sales.csv`;
```

Note:

- CSV requires full file path
- Parquet can be read at folder level

Task 8: Explore DBUtils

View Help

```
dbutils.help()
```

List Files

```
dbutils.fs.ls("/Volumes/databricks-abic/bronze/bronze_volume/sales/")
```

Create Directory

```
dbutils.fs.mkdirs("/Volumes/databricks-abic/bronze/bronze_volume/test/")
```

Create Widget

```
dbutils.widgets.text("parameter1", "")
```

🎯 Outcome

After completing this lab, you:

- Created custom catalog via UI
- Understood catalog-level configuration
- Created Bronze schema
- Created managed table
- Created managed volume
- Built raw folder structure
- Queried CSV data from volume
- Used DBUtils utilities
- Prepared Bronze layer properly